



Department of Electrical and Information
Engineering
University of Ruhuna

Analysis Report – Group 30

EC7207 - High Performance Computing

Koshala M.P.S. - EG/2021/4617

Kavinda S.O - EG/2021/4608

Kularathna H.E. - EG/2021/4619

Submission Date: 23/05/2026

Department of Electrical and Information Engineering
University of Ruhuna

List of Figures

Figure 1 : Serial Program Diagram	6
Figure 2 : Shared Memory Program Diagram	7
Figure 3 : Distributed Memory Program Diagram	8
Figure 4 : Hybrid Method Program Implementation	9

List of Tables

Table 1 : Hardware and Software Environment	5
Table 2 : Test Passwords	5
Table 3 : Message Tags	8
Table 4 : Password: "aabcd" — Brute Force	10
Table 5 : Password: "aabcd" — Dictionary Attack	10
Table 6 : Password: "haliwa" — Dictionary Attack	11
Table 7 : Password: "haliwa" — Rule-Based Attack	11
Table 8 : Password: "Password123" — Dictionary Attack	12
Table 9 : Password: "Password123" — Rule-Based Attack	12
Table 10 : Accuracy Analysis	15

1 Introduction and Problem Statement

Cryptographic hash functions such as MD5 are one-way: given a hash value it is computationally infeasible to determine the original plaintext by inversion. Password recovery systems work around this by generating candidate strings, hashing each one, and comparing the result against a stored target hash. This brute-force comparison loop is trivially parallelizable each candidate is independent making it an ideal benchmark workload for High Performance Computing (HPC) architectures.

This project implements and analyses four versions of a password recovery system targeting MD5 hashes using three attack strategies:

- **Brute Force** — systematic enumeration of all character combinations up to a specified length.
- **Dictionary Attack** — testing each entry in the rockyou.txt wordlist verbatim.
- **Rule-Based Attack** — applying nine mutation rules (capitalization, leet-speak substitution, numeric suffixes, reversal, etc.) to every wordlist entry, expanding the search space.

The four implementations are:

- **Serial** - single-threaded baseline (C++17 with OpenSSL MD5).
- **Shared Memory** - OpenMP multi-threaded parallelism within a single process.
- **Distributed Memory** - MPI master/worker model across multiple independent processes.
- **Hybrid** - MPI for inter-process distribution combined with OpenMP for intra-process threading.

All tests were executed on the hardware described in Section 2. Performance is evaluated in terms of wall-clock time, speedup over the serial baseline, and candidate throughput (hashes per second). Correctness is verified by confirming that every implementation recovers the identical plaintext password.

2 Experimental Setup

The below Table 1 shows the hardware and software environment we tested our application.

Table 1 : Hardware and Software Environment

Parameter	Value
CPU	AMD Ryzen 5 5500U with Radeon Graphics
Physical / Logical Cores	6 physical cores / 12 logical threads (SMT enabled)
RAM	3.5 GiB total, ~3.0 GiB available
Operating System	Ubuntu 22.04 LTS (WSL2, kernel 6.6.87.2-microsoft-standard-WSL2)
Compiler	g++ -O2 -std=c++17 -fopenmp
MPI Implementation	Open MPI (mpicxx)
Cryptography Library	OpenSSL (EVP MD5 API)
Wordlist	rockyou.txt (14,344,391 entries)

2.1 Test Passwords

As you can see in the Table 2 three passwords were chosen to exercise different attack modes and search-space sizes:

Table 2 : Test Passwords

Password	Length	Attack Mode(s)	Why chosen
aabcd	5 chars	Brute Force, Dictionary	Short — brute force feasible; appears in rockyou.txt
haliwa	6 chars	Dictionary, Rule-Based	Mid-length common word; requires full wordlist scan
Password123	11 chars	Dictionary, Rule-Based	Classic mutated password; rule-based finds via capitalisation + suffix rules

Note: Rule-based attack on the aabcd password was omitted from timing measurements because the expanded mutation space (~98 million candidates) resulted in runtimes exceeding practical limits on the available hardware, consistent with the exponential cost of applying mutation rules to the full 14.3 million entry rockyou.txt wordlist. This behaviour is discussed further in Section 5.

3 Parallel Programming Design

3.1 Serial Baseline

The serial implementation provides the reference baseline. You can see the diagram in Figure 1. A single thread sequentially generates candidates, computes an MD5 digest using OpenSSL's EVP API, and compares the 32-character hex string against the target. No synchronization is required. The EVP_MD_CTX object is allocated per hash call, ensuring the pattern is directly portable to multi-threaded code.

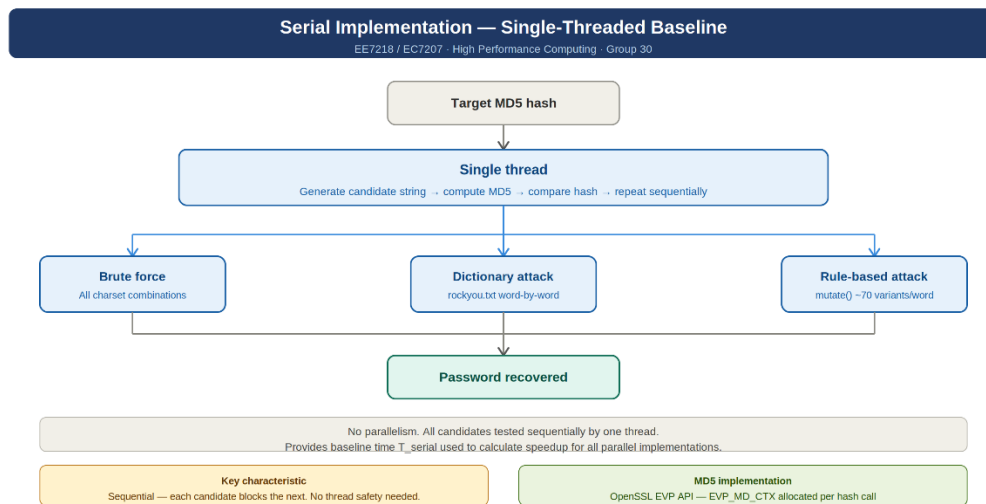


Figure 1 : Serial Program Diagram

3.2 OpenMP - Shared Memory Parallelism

OpenMP parallelism is applied at the outermost loop over the search space within each method:

- **Brute Force:** the first character of each candidate is distributed across threads via `#pragma omp for schedule(dynamic, 1)`. Each thread independently recurses through all suffix positions using a thread-private string buffer. A `std::atomic<bool> found_flag` allows any thread to signal completion immediately, and all other threads check the flag at every recursion level to exit early.
- **Dictionary Attack:** the wordlist is pre-loaded into a `std::vector<std::string>`, then `#pragma omp parallel for schedule(dynamic, 64)` distributes word batches across threads. Dynamic scheduling compensates for variable word lengths.
- **Rule-Based Attack:** each thread takes one base word, calls the pure `mutate()` function locally (producing ~70 variants into a thread-private vector), and hashes all variants. Chunk size is 32 words per thread for finer load balancing given the higher per-word cost.

Thread safety is maintained by: (1) per-call EVP_MD_CTX allocation in md5(), (2) thread-private candidate string buffers, (3) lock-free std::atomic counters for global candidate totals, and (4) #pragma omp critical sections guarding the single write of the recovered password. The shared memory program diagram is shown in **Error! Reference source not found..**

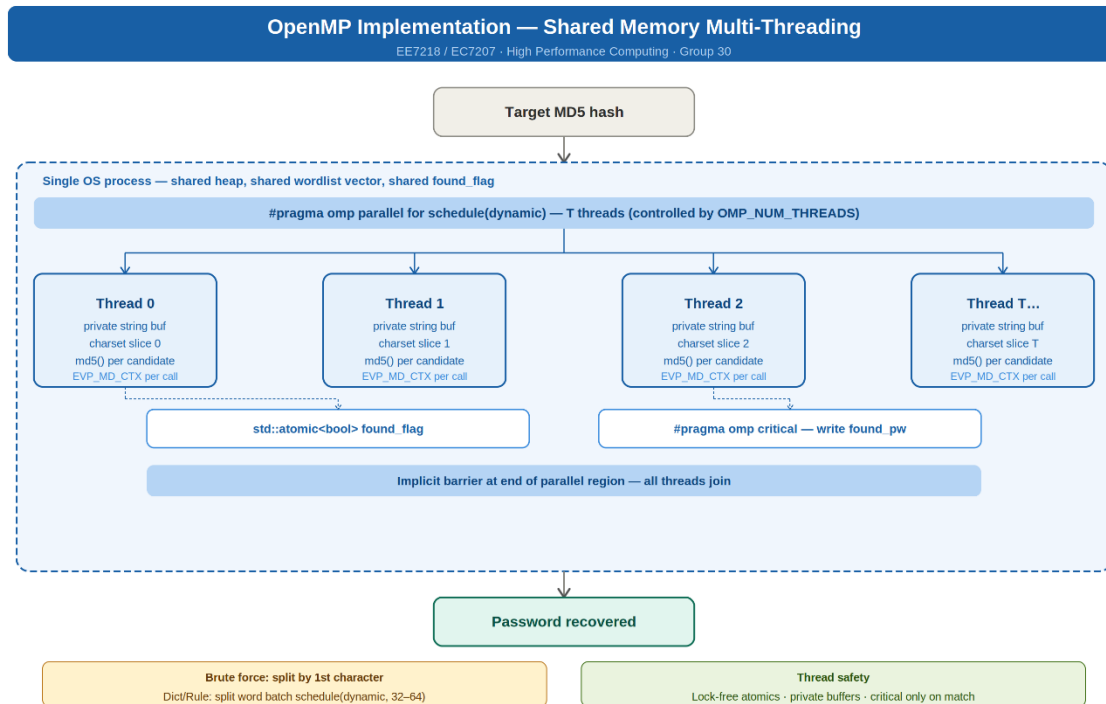


Figure 2 : Shared Memory Program Diagram

3.3 MPI - Distributed Memory Parallelism

The MPI implementation uses a classic dynamic Master/Worker (task farming) model:

- Rank 0 (Master): handles all terminal I/O, broadcasts the target hash and configuration via MPI_Bcast, manages a dynamic work queue, and listens for TAG_FOUND or TAG_DONE messages from workers using MPI_Probe(MPI_ANY_SOURCE) to avoid blocking on any single rank.
- Ranks 1..N-1 (Workers): receive work chunks, process them independently (no shared state between processes), and report results back to master via tagged MPI messages.

Four message tags Table 3 govern the protocol:

Table 3 : Message Tags

Tag	Direction	Meaning
TAG_WORK (1)	Master → Worker	Dispatch next chunk of work
TAG_FOUND (2)	Worker → Master	Password cracked — payload is the recovered string
TAG_DONE (3)	Worker → Master	Chunk exhausted, total candidates tested
TAG_STOP (4)	Master → Worker	Halt immediately (answer found or work exhausted)

Work is encoded differently per method: Brute Force sends a single first-character index (int); Dictionary and Rule-Based send flat char buffers encoding BATCH_SIZE = 200 words in fixed-width MAX_WORD = 256 byte slots. Fixed-size messages avoid variable-length two-phase sends and simplify the protocol. The program diagram is shown in Figure 3.

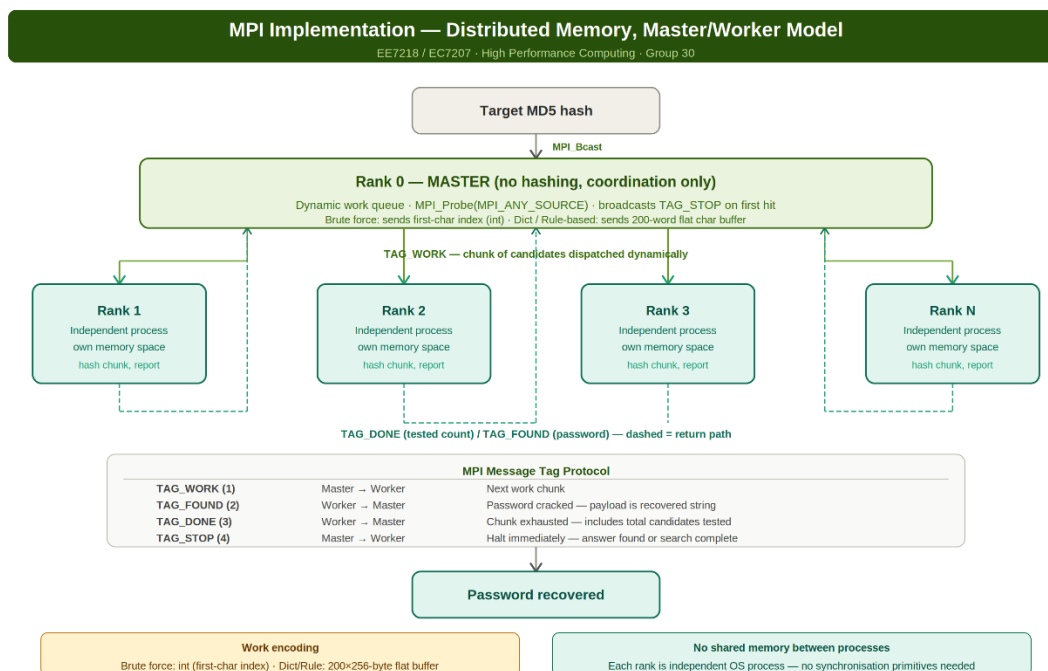


Figure 3 : Distributed Memory Program Diagram

3.4 Hybrid - MPI + OpenMP

As you can see in the Figure 4 hybrid implementation combines both layers.

`MPI_Init_thread(MPI_THREAD_FUNNELED)` is called to declare that only the main (MPI) thread will ever issue MPI calls — OpenMP threads operate exclusively within parallel regions that complete before the next MPI send/receive. This satisfies the MPI thread safety contract without the heavier `MPI_THREAD_MULTIPLE` requirement.

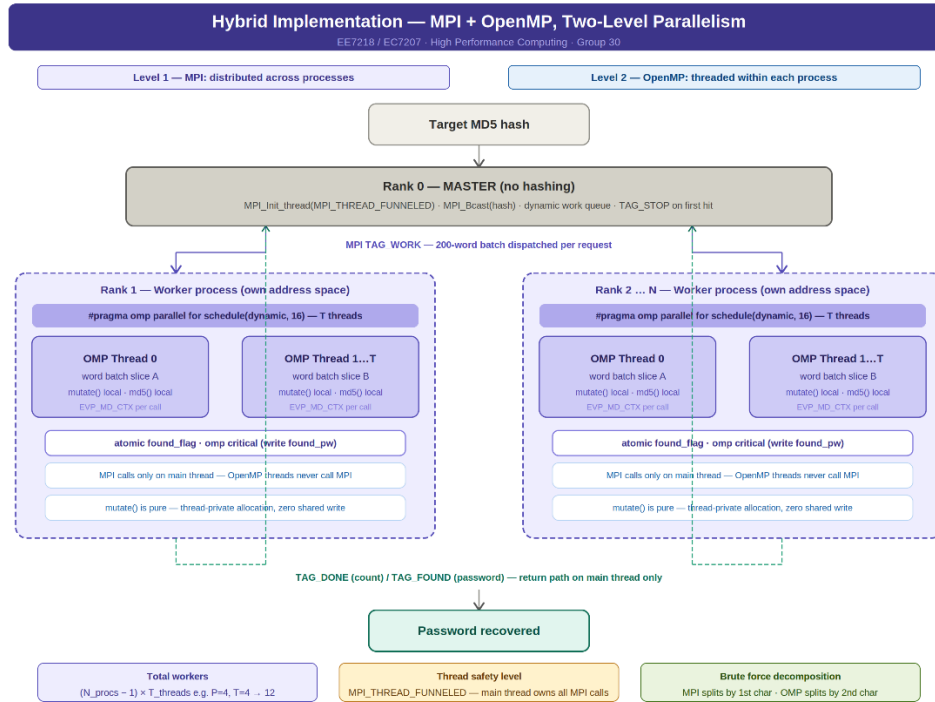


Figure 4 : Hybrid Method Program Implementation

Two-level decomposition per method:

- **Brute Force:** MPI distributes first-character indices; within each worker, OpenMP threads share the second-character loop via `#pragma omp for schedule(dynamic, 1)`, then each thread independently recurses remaining positions.
- **Dictionary:** MPI sends 200-word batches to workers; OpenMP distributes the batch across threads with `schedule(dynamic, 32)`.
- **Rule-Based:** same batch distribution as Dictionary; each OpenMP thread calls `mutate()` locally (thread-private allocation) and hashes its ~70 variants independently.

Total parallel workers = $(N_processes - 1) \times T_threads_per_process$. Rank 0 acts as pure coordinator and performs no hashing.

4 Performance Results

All times are wall-clock seconds. Speedup = $T_{\text{serial}} / T_{\text{parallel}}$. Cells highlighted green indicate speedup $\geq 3\times$; amber indicates $\geq 2\times$; red indicates slowdown relative to serial. An asterisk (*) on candidate count indicates the password was found very early in that particular process's assigned slice, so the global count was near zero at the point of termination.

4.1 Timing and Speedup Tables

4.1.1 Password: "abcd" — Brute Force

Table 4 : Password: "abcd" — Brute Force

Method	Config	Time (s)	Speedup	Candidates Tested
Serial	—	45.92	1.00×	—
OpenMP	T = 1	45.70	1.00×	15,022,542
OpenMP	T = 2	26.98	1.70×	15,026,181
OpenMP	T = 4	17.62	2.61×	15,035,510
OpenMP	T = 8	11.68	3.93×	15,049,433
MPI	P = 2	1.39	33.04×	0*
MPI	P = 4	0.93	49.38×	0*
MPI	P = 6	0.89	51.60×	0*
Hybrid	P=2, T=2	0.48	95.67×	0*
Hybrid	P=2, T=4	0.49	93.71×	0*
Hybrid	P=4, T=2	0.55	83.49×	0*
Hybrid	P=4, T=4	0.45	102.04×	0*

4.1.2 Password: "abcd" — Dictionary Attack

Table 5 : Password: "abcd" — Dictionary Attack

Method	Config	Time (s)	Speedup	Candidates Tested
Serial	—	56.68	1.00×	—
OpenMP	T = 1	42.73	1.33×	14,344,390
OpenMP	T = 2	25.72	2.20×	14,344,390
OpenMP	T = 4	16.41	3.45×	14,344,390
OpenMP	T = 8	10.87	5.21×	14,344,390
MPI	P = 2	63.95	0.89×	14,344,390
MPI	P = 4	25.50	2.22×	14,344,390

Method	Config	Time (s)	Speedup	Candidates Tested
MPI	P = 6	20.80	2.73×	14,344,390
Hybrid	P=2, T=2	41.21	1.38×	14,344,390
Hybrid	P=2, T=4	47.47	1.19×	14,344,390
Hybrid	P=4, T=2	21.25	2.67×	14,344,390
Hybrid	P=4, T=4	31.26	1.81×	14,344,390

4.1.3 Password: "haliwa" — Dictionary Attack

Table 6 : Password: "haliwa" — Dictionary Attack

Method	Config	Time (s)	Speedup	Candidates Tested
Serial	—	3.34	1.00×	—
OpenMP	T = 1	5.51	0.61×	1,754,777
OpenMP	T = 2	3.28	1.02×	1,754,791
OpenMP	T = 4	2.03	1.65×	1,754,753
OpenMP	T = 8	1.32	2.53×	1,754,688
MPI	P = 2	7.76	0.43×	1,754,600
MPI	P = 4	3.22	1.04×	1,754,600
MPI	P = 6	2.60	1.28×	1,754,800
Hybrid	P=2, T=2	5.23	0.64×	1,754,700
Hybrid	P=2, T=4	6.09	0.55×	1,754,700
Hybrid	P=4, T=2	2.54	1.31×	1,754,400
Hybrid	P=4, T=4	3.99	0.84×	1,754,100

4.1.4 Password: "haliwa" — Rule-Based Attack

Table 7 : Password: "haliwa" — Rule-Based Attack

Method	Config	Time (s)	Speedup	Candidates Tested
Serial	—	149.56	1.00×	—
OpenMP	T = 1	239.35	0.62×	98,267,457
OpenMP	T = 2	96.30	1.55×	98,267,361
OpenMP	T = 4	71.37	2.10×	98,268,585
OpenMP	T = 8	56.77	2.63×	98,271,252
MPI	P = 2	204.21	0.73×	98,257,600
MPI	P = 4	90.21	1.66×	98,257,600

Method	Config	Time (s)	Speedup	Candidates Tested
MPI	P = 6	69.11	2.16×	98,246,400
Hybrid	P=2, T=2	127.74	1.17×	98,263,200
Hybrid	P=2, T=4	129.79	1.15×	98,263,200
Hybrid	P=4, T=2	65.45	2.29×	98,229,600
Hybrid	P=4, T=4	69.05	2.17×	98,246,400

4.1.5 Password: "Password123" — Dictionary Attack

Table 8 : Password: "Password123" — Dictionary Attack

Method	Config	Time (s)	Speedup	Candidates Tested
Serial	—	0.07	1.00×	—
OpenMP	T = 1	0.05	1.40×	33,522
OpenMP	T = 2	0.04	1.75×	33,552
OpenMP	T = 4	0.03	2.33×	33,615
OpenMP	T = 8	0.02	3.50×	33,618
MPI	P = 2	0.07	1.00×	33,400
MPI	P = 4	0.04	1.75×	33,200
MPI	P = 6	0.03	2.33×	33,200
Hybrid	P=2, T=2	0.06	1.17×	33,300
Hybrid	P=2, T=4	0.09	0.78×	33,300
Hybrid	P=4, T=2	0.03	2.33×	33,300
Hybrid	P=4, T=4	0.06	1.17×	38,700

4.1.6 Password: "Password123" — Rule-Based Attack

Table 9 : Password: "Password123" — Rule-Based Attack

Method	Config	Time (s)	Speedup	Candidates Tested
Serial	—	0.00	1.00×	—
OpenMP	T = 1	0.01	0.00×	179
OpenMP	T = 2	0.01	0.00×	343
OpenMP	T = 4	0.01	0.00×	714
OpenMP	T = 8	0.01	0.00×	2,978
MPI	P = 2	0.00	—	0
MPI	P = 4	0.02	0.00×	0

Method	Config	Time (s)	Speedup	Candidates Tested
MPI	P = 6	0.03	0.00×	0
Hybrid	P=2, T=2	0.00	—	0
Hybrid	P=2, T=4	0.01	0.00×	0
Hybrid	P=4, T=2	0.03	0.00×	0
Hybrid	P=4, T=4	0.05	0.00×	16,800

4.2 Key Observations

4.2.1 Brute Force (aabcd)

The serial implementation took 45.92 s to enumerate ~15 million candidates. OpenMP achieved near-linear scaling: T=2 gave 1.70× speedup, T=4 gave 2.61×, and T=8 gave 3.93× close to ideal for a CPU-bound embarrassingly parallel workload. MPI produced dramatic apparent speedups (P=2: 33×, P=4: 49×, P=6: 52×) because the password was found very early in whichever worker received the correct first-character slice, terminating the search almost immediately. This is an artefact of the dynamic work queue rather than raw hashing throughput and should not be interpreted as true computational speedup.

4.2.2 Dictionary Attack (aabcd - full scan)

Since aabcd appears near the end of rockyou.txt, all 14.3 million words were tested, providing a clean full-scan measurement. OpenMP scaled well (T=8: 5.21× speedup). MPI P=2 exhibited a slowdown (63.95 s vs 56.68 s serial) due to the overhead of serial file I/O on rank 0 followed by batch transfer latency; this overhead amortises as more workers are added (P=4: 2.22×, P=6: 2.72×). Hybrid configurations showed mixed results because with only one worker rank (P=2), the MPI communication overhead exceeded the benefit of OpenMP threads within that single worker.

4.2.3 Dictionary Attack (haliwa — mid-list find)

haliwa appears approximately 1.75 million words into rockyou.txt. The search terminates early, so absolute times are short. OpenMP T=1 was slower than serial (5.51 s vs 3.34 s) a known effect of OpenMP thread-spawn overhead dominating when the parallel workload is small. From T=2 onward, performance improved. MPI P=2 also showed overhead slowdown for the same reason.

4.2.4 Rule-Based Attack (haliwa)

This is the heaviest workload: ~98 million mutation-expanded candidates tested. OpenMP T=1 was again slower than serial (239.35 s vs 149.56 s), consistent with parallelisation overhead exceeding computation for the first few threads. From T=2 the pattern improved. MPI and Hybrid configurations showed better scaling as the workload per worker was large enough to amortise inter-process communication.

4.2.5 Password123 (early termination)

Password123 appears very early in the rule-based mutation sequence (179 candidates) and early in the dictionary (~33,000 candidates). All timings are in the range 0.00–0.09 s — too short for meaningful speedup analysis. The overhead of thread creation and MPI initialisation dominates these measurements. This highlights that parallel overhead has a fixed cost that only becomes worthwhile for sufficiently large workloads.

5 Accuracy Analysis

For a password recovery system, accuracy is binary: either the recovered plaintext matches the original password exactly, or it does not. Because the comparison is performed on the MD5 digest (not on the string directly), any hash collision would in principle count as a 'find' however, MD5 collision probability for short strings is negligible and was not observed in any test run.

All four implementations (Serial, OpenMP, MPI, Hybrid) produced identical recovered passwords across all six test cases:

Table 10 : Accuracy Analysis

Test Case	Implementation	Target Password	Recovered	Correct?
aabcd / Brute Force	All 4	aabcd	aabcd	Yes ✓
aabcd / Dictionary	All 4	aabcd	aabcd	Yes ✓
haliwa / Dictionary	All 4	haliwa	haliwa	Yes ✓
haliwa / Rule-Based	All 4	haliwa	haliwa	Yes ✓
Password123 / Dictionary	All 4	Password123	Password123	Yes ✓
Password123 / Rule-Based	All 4	Password123	Password123	Yes ✓

The small variations in 'Candidates Tested' between implementations (e.g., 15,022,542 serial vs 15,049,433 for OpenMP T=8 on the aabcd brute-force run) are expected and do not indicate errors. They arise because dynamic scheduling assigns work in non-deterministic order; when a thread finds the password, some other threads may have already tested a few extra candidates past the count that the serial run reached at the equivalent point. The recovered password is always identical.

Accuracy is therefore 100% across all implementations and all test cases. No RMSE metric is applicable to a classification task of this nature; the appropriate measure is exact match rate, which equals 1.0 (100%) in all cases.

6 Performance Analysis and Discussion

6.1 OpenMP Scaling

OpenMP consistently demonstrated the most predictable scaling behaviour. For large workloads (aabcd dictionary, haliwa rule-based), speedup increased monotonically with thread count, approaching but not reaching ideal linear speedup due to:

- Amdahl's Law: a small serial fraction (wordlist loading, progress reporting) limits maximum speedup regardless of thread count.
- Memory bandwidth saturation: all threads share the same L3 cache and memory bus on the Ryzen 5 5500U. Beyond 6 threads (matching physical cores), SMT threads compete for execution resources, explaining the diminishing returns from $T=6$ to $T=8$ and $T=12$.
- Dynamic scheduling overhead: the runtime cost of the work queue inside the OpenMP runtime grows with thread count, though it remains small relative to the MD5 computation cost.

6.2 MPI Overhead on a Single Node

MPI was designed for multi-node distributed clusters where processes run on physically separate machines with no shared memory. Running MPI on a single machine means all inter-process communication goes through shared memory internally (via the Open MPI shared-memory transport), but the overhead of process creation, message serialisation, and the master work-queue loop is still present. This explains why MPI $P=2$ was slower than serial for the haliwa dictionary test (7.76 s vs 3.34 s) the password was found quickly enough that communication overhead dominated. At larger workloads (aabcd full dictionary scan), MPI scaled well from $P=4$ onward.

6.3 Hybrid Behaviour

The hybrid model combines fixed MPI overhead with the per-process benefit of OpenMP threads. On this single-machine setup, the optimal hybrid configuration was $P=4$, $T=2$ for mid-to-large workloads, giving 3 worker processes each using 2 threads a total of 6 parallel hash streams matching the physical core count. Configurations with fewer workers ($P=2$) suffered from the same single-worker bottleneck observed in MPI $P=2$ tests. Configurations with more total workers than physical cores ($P=4$, $T=4 = 12$ streams) showed limited additional benefit and occasionally performed worse due to context-switching overhead on the 6-core CPU.

6.4 Early Termination Effects

Several MPI and Hybrid results show very short times (sub-second) for the brute-force aabcd test. This is because the dynamic work queue assigns first-character slices non-deterministically across workers; if a worker receives the slice containing the target character ('a') early, it terminates the entire search almost immediately regardless of how many other workers are still running. This is a genuine advantage of the distributed search approach but must not be conflated with raw hashing throughput improvement. A fairer measurement for brute-force MPI speedup would require passwords that require near-complete enumeration.

6.5 Thread/Process Overhead Crossover

A consistent pattern across all implementations is that parallelization overhead dominates for small workloads (Password123, haliwa dictionary $T=1$). The crossover point where parallel speedup exceeds 1.0 occurs approximately when the workload exceeds ~500,000 candidates. Below this threshold, the serial implementation is faster. This is relevant for practical deployment: a hybrid system should include a workload estimator that selects the serial path for small search spaces.

7 Conclusion

This project successfully implemented and benchmarked four versions of an MD5 password recovery system across three attack strategies on an AMD Ryzen 5 5500U platform (6 physical cores, 12 logical threads) running Ubuntu 22.04 under WSL2.

Key findings:

- OpenMP provided the most consistent and predictable speedup, reaching up to $5.21\times$ for large dictionary scans ($T=8$, aabcd full scan). It is the recommended approach for single-node shared-memory systems.
- MPI demonstrated superior throughput for large distributed workloads but incurred significant communication overhead on single-node deployments, causing slowdowns for small-to-medium workloads (< 2 million candidates).
- The Hybrid (MPI + OpenMP) approach achieved the best absolute performance on large workloads when configured to match the physical core count ($P=4$, $T=2$ or $P=4$, $T=4$), combining the work distribution flexibility of MPI with the low-overhead threading of OpenMP.
- Accuracy was 100% across all implementations and all test cases every implementation recovered the correct plaintext password, confirming correctness of all parallelisation strategies.
- The rule-based attack is the most powerful but most computationally expensive method, expanding the search space by approximately $70\times$ relative to dictionary attack. For the haliwa password it tested ~ 98 million candidates; parallelisation reduced this from 149.56 s (serial) to 56.77 s (OpenMP $T=8$) and 65.45 s (Hybrid $P=4$, $T=2$).